Desafio empresa Primora

Parte 1 - Tratamento dos Dados

O contexto, e demais dados da primeira parte, foram enviados pela própria empresa e estão nos arquivos fornecidos por eles:

02 - Desafio_1_Backend_ETL.pdf

02 - LEIAME.pdf

02-clientes_distribuidora_vetor.csv

Além do tratamento dos dados solicitado nos documentos, o grupo também deverá criar uma API para consumo do banco de dados.

Parte 2 - Projeto — API de Consumo de Dados da Distribuidora Vetor

Contexto

Após a etapa de ETL e normalização dos dados de clientes da Distribuidora Vetor, os registros foram carregados em um banco relacional. Agora, a equipe precisa disponibilizar esses dados por meio de uma **API REST em Django**, permitindo consultas públicas e operações restritas a usuários autenticados.

Objetivos

- **CRUD de clientes**: listar, consultar, criar, editar e excluir registros.
- **CRUD de vendedores**: gerenciar tabela de vendedores responsáveis.
- **Consultas por bairro**: rota que agrupe clientes por bairro normalizado.
- **Consultas por vendedor**: rota que liste clientes atendidos por cada vendedor.
- **Autenticação**: login via Django Auth + Token Authentication (DRF).
- **Controle de acesso**:
    - Usuários **logados** → CRUD completo.
    - Usuários **não logados** → apenas GET.
- **Documentação**: gerar schema OpenAPI com drf-spectacular e disponibilizar em Redoc.

Estrutura de Dados

A API deve refletir as tabelas criadas no banco após o ETL:

1. **Cliente**
    - id_cliente
    - razao_social
    - cnpj_normalizado
    - telefone_normalizado

    - email
    - bairro_normalizado
    - cep_normalizado
    - data_cadastro_normalizado
    - vendedor_id (FK → Vendedor)
2. **Vendedor**
    - id_vendedor
    - nome_vendedor

(Tabela de Auditoria permanece apenas no admin, não exposta na API.)

Rotas mínimas

- `/api/clientes/` → CRUD de clientes.
- `/api/clientes/{id}/` → detalhes de cliente específico.
- `/api/clientes/bairro/{nome}/` → listar clientes por bairro.
- `/api/clientes/vendedor/{id}/` → listar clientes por vendedor.
- `/api/vendedores/` → CRUD de vendedores.
- `/api/vendedores/{id}/clientes/` → listar clientes de um vendedor.
- `/api/auth/token/` → autenticação via token.
- `/api/docs/` → documentação interativa.

Requisitos

- CRUD completo para clientes e vendedores.
- Pelo menos 2 rotas de relacionamento (clientes ↔ bairro, clientes ↔ vendedor).
- Autenticação com DRF Token.
- Controle de acesso: logados podem alterar, não logados apenas consultar.
- Registro no admin de todas as tabelas.
- Documentação clara da API.
- # Entregáveis
    - Código fonte
    - Repositório GitHub organizado.
    - Modelos, rotas, controllers/services.
    - Arquivo de dependências.
    - Scripts de migração.
    - Estrutura de pastas limpa.

Documentação

- README com:
  - descrição do sistema
  - criação de ambiente
  - instalação de pacotes
  - instruções de instalação
  - como rodar o servidor
- Documentação da API (Markdown ou Swagger).
- Modelo de dados (MER + DER).